ODSC AI EAST 2026

Data Engineering

DEPLOYING A SECURE AUTONOMOUS AGENT FOR REAL-TIME COMPLIANCE MONITORING

SCOTT ASKINOSIE

Developer Advocate

Contextual AI

ODSC AI EAST 2026

Feel free to connect with us on LinkedIn!

Scott Askinosie M.S. Ph.D.
Principal AI Engineer
Western Governors

Jonathan Brown
Principle Machine Learning Engineer
Western Governors University

Compliance changes.
Quietly.
Constantly.

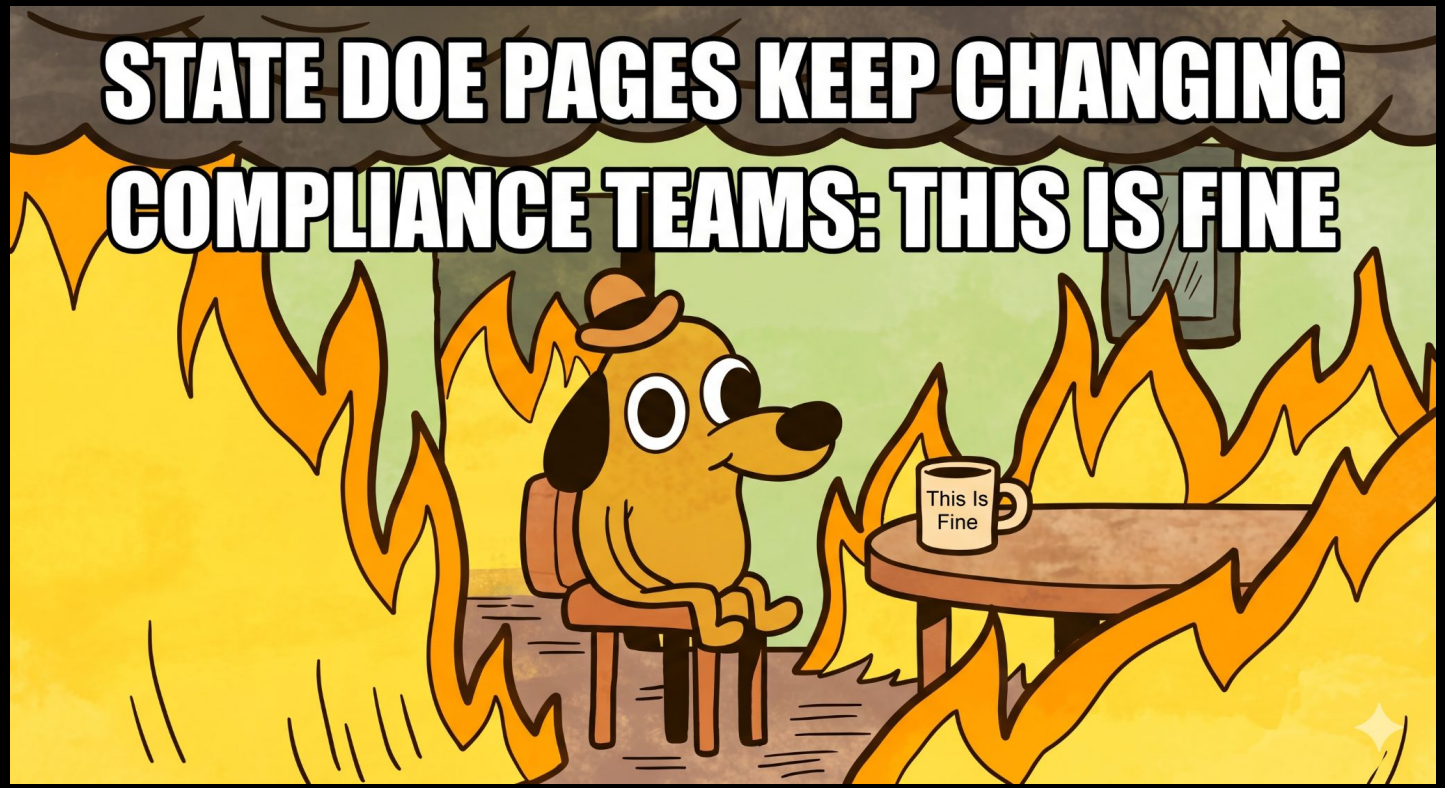
Universities in the USA MUST train teachers!!!

- Different States have different licensure
  - No central notification system
  - Dozens of individual state agencies
- Tracking is highly disconnected and significant administrative burden

50	∞	0	0	0
STATES	PAGE EDITS	WEBHOOKS	RSS FEEDS	CENTRAL DASHBOARDS

ODSC AI EAST 2026

ODSC AI EAST 2026



ODSC AI EAST 2026

We build it twice. Then we watch it work.

1

CUSTOM CLAW

build it from scratch

pydantic-ai agent
add skills one at a time

2

OPENCLAW

switch to the framework

production grade
Docker, dashboard, channels

3

OPIK

see what it's doing

trace every turn
LLM spans, tool calls, costs

Custom Claw

build it from scratch

(spoiler: it's not as hard as you think)

ODSC AI EAST 2026

first principles

LLM vs Agent

JUST AN LLM

- ✖ knows a lot from training
- ✖ can reason about what it knows
- ✖ can't browse the web today
- ✖ no memory between calls
- ✖ can't take action in the world

great at trivia. useless for compliance.

AN AGENT

- ✓ an LLM + skills it can call
- ✓ fetches live pages
- ✓ stores state across runs
- ✓ decides which skill to call when
- ✓ you don't write the routing logic

the difference is one decorator.

let's start with what we have

Step 1: A raw LLM call.

ask gpt-4o-mini about Ohio Praxis. it'll answer — from training data.

step_1.py

```
from openai import OpenAI

client = OpenAI()

response = client.chat.completions.create(
    model="gpt-4o-mini",
    messages=[{"role": "user", "content":
        "What Praxis tests are required for Math 7-12 in Ohio?"
    }]
)
```

the catch: *training data is a snapshot. The Ohio DOE page may have changed yesterday. The LLM has no idea.*

LLM vs Agent

now we give it a tool

Step 2: One line, one skill.

step_2.py

```python
from pydantic_ai import Agent
from skills.fetch_page import fetch_compliance_page

agent = Agent(
    model="openai:gpt-4o-mini",
    instructions="You monitor compliance pages..."
)

agent.tool_plain(fetch_compliance_page)  # ← that's it

result = await agent.run(
    "Fetch the Ohio DOE licensure page and tell me what it says."
)
```

one decorator. now your LLM can read live web pages.

wait. who wrote that fetch function?

The LLM did.

ask the same model that powers the agent to write its own skill code.
no separate tool. no copy-paste. one prompt, one function.

```
response = client.chat.completions.create(
    model="gpt-4o-mini",
    messages=[{"role": "user", "content":
        "Write a Python function fetch_compliance_page(url: str) -> str
         that fetches a page and returns cleaned text.
         Strip HTML tags, scripts, styles. Collapse whitespace."
    }]
)
```

and that's basically what's in skills/fetch_page.py.
(we added SSRF protection on top — but the core logic? gpt wrote it.)

every new capability is one function

Step 3: More skills.

each lives in skills/. one file = one capability. the agent decides when to call each.

`fetch_page`

SSRF-safe HTTP GET. pinned IPs, no rebinding.

`check_change`

SHA-256 the body. diff against SQLite snapshot.

`discover_urls`

Brave Search for compliance pages by state.

`send_alert`

message with state + URL + summary.

`agent.tool_plain(fetch_compliance_page)` · `agent.tool_plain(check_for_change)` · `agent.tool_plain(discover_urls)` · `agent.tool_plain(send_alert)`

stage 1, complete. but...

The ceiling.

custom claw works. adding a skill is one file + one line. that's the easy part.

here's everything you'd own forever:

SSRF-safe URL fetching	*IP pinning, scheme allowlist, redirect handling*
SQLite snapshot store	*schema, migrations, query layer*
Tool routing loop	*agent loop orchestration, error retry, tool ordering*
Networking + auth + errors	*timeouts, rate limits, key rotation, secrets*
Scheduling & persistence	*you're cron-ing or running notebooks by hand*

you wrote it. you maintain it. forever.

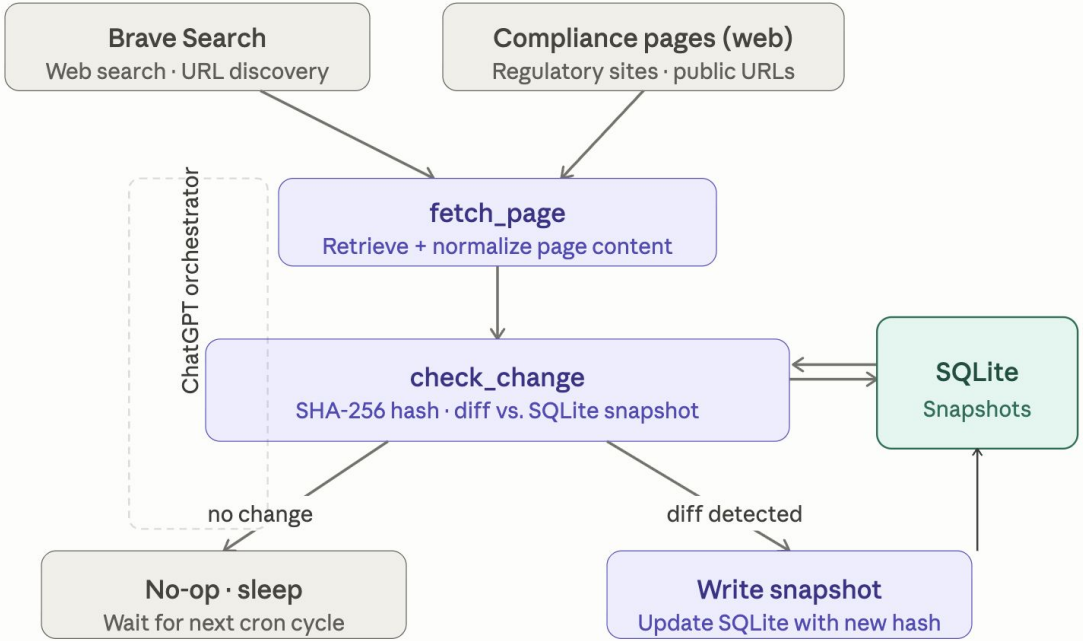
Lesson 1 Building our own “Custom Claw”

Pydantic AI

ODSC AI EAST 2026

Custom Claw — build it yourself

ChatGPT + explicit function calls



Tradeoffs:

✓ Readable end-to-end · easy to debug

✕ No memory · new feature = more Python

but...

There's a better way.

stage 2

OpenClaw

the same agent. but production.

Docker. dashboard. always-on. multi-channel. no plumbing.

AGENTS.md

with Custom Claw, domain knowledge lives in code, with OpenClaw, you externalize it.

AGENTS.md

```
# Compliance Monitor Agent — Context

## Domain Knowledge
- Every US state sets its own educator certification requirements
- Praxis exams administered by ETS — Core + Subject Assessments

## Skills Available
- brave-search: web search for licensure news, district pages
- check-compliance-updates: diff monitored URLs, log changes

## Behavior Guidelines
- Always cite the state and URL when reporting compliance info
- Never state requirements as permanent — note the source date
```

AGENTS.md tells it WHAT it knows.
Skills tell it WHAT it can do.

`AGENTS.md`

the brain

- domain knowledge
- key sources by state
- behavior guidelines
- which skills exist & when

`skills/*/SKILL.md`

the hands

- tool definitions
- when to use this skill
- when NOT to
- shell commands to run

skills are markdown. and bash.

Skills = Markdown + script.

the skill description tells the agent when to use it. the script is what runs. no Python required.

openclaw-agent/skills/check-compliance-updates/SKILL.md

```
---
name: check-compliance-updates
description: "Check for teacher licensure and Praxis exam
   requirement updates for a given US state.
   Use when: asked about licensure changes,
   Praxis requirements, certification updates.
   NOT for: news unrelated to certification."
---

## When to Use
✅ "Check for licensure updates in Ohio"
✅ "Have Praxis requirements changed in Texas?"

## Commands
bash /app/skills/check-compliance-updates/check.sh "Ohio"
```

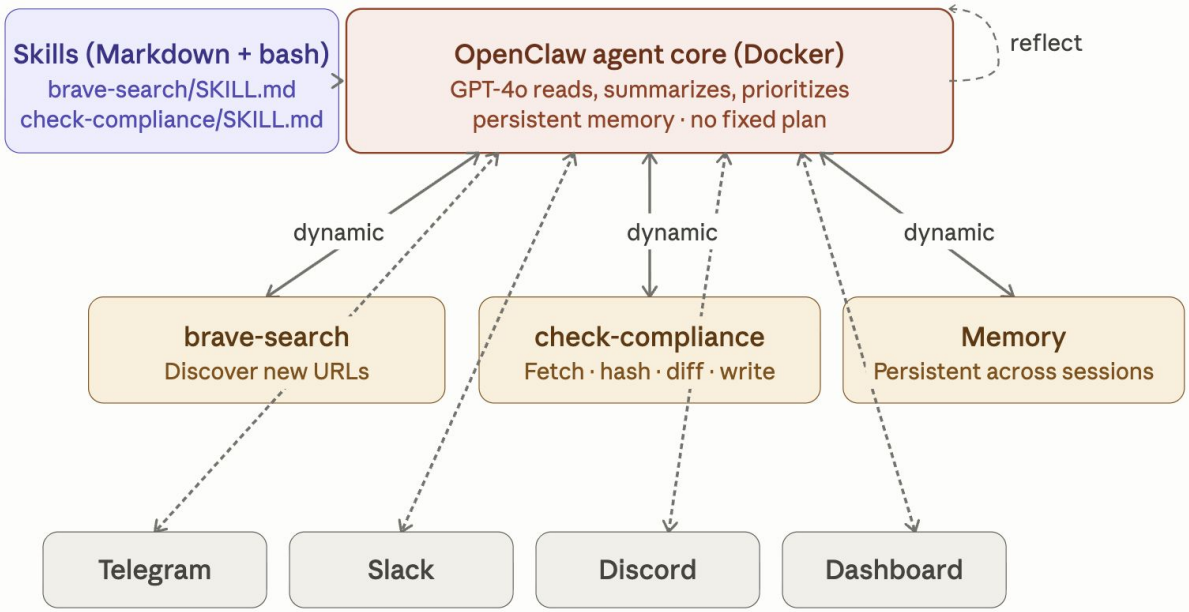
Side by side.

	CUSTOM CLAW	OPENCLAW
framework	pydantic-ai	OpenClaw + Docker
skills	Python functions + decorator	Markdown + shell scripts
domain knowledge	hardcoded in agent.py	AGENTS.md (read every turn)
URL discovery	Brave Search skill	same Brave skill, plus web dashboard
channels	Telegram via .env (manual setup)	Telegram pairing code, plus more
scheduling	you run the notebook	built-in: "every Monday at 9am..."
memory	SQLite snapshots only	persistent context across sessions
who maintains it	you	the framework

the framework adds operability. you keep the domain.

OpenClaw — use a framework

Skills in Markdown + bash · framework handles the rest



What you get for free:

✓ Persistent memory ✓ Multi-channel ✓ Web dashboard ✓ Scheduler built-in ✓ Skills in Markdown

ODSC AI EAST 2026

now you can text your agent anything

Things you can say:

COMPLIANCE

"Check Ohio for licensure updates"

"Have Praxis requirements changed in TX?"

"What changed this week?"

SCHEDULED TASKS

"Every Monday at 9am, check all states"

"Send me a daily compliance digest"

"Remind me Friday to review renewals"

ON-THE-FLY SKILLS

"Create a skill that monitors NASDTEC"

"Write a skill that checks ETS Praxis scores"

"Add a skill for state board minutes"

yes — "create a skill" actually works. AGENTS.md teaches it how.

Lesson 2 The Claw

OpenClaw

ODSC AI EAST 2026

stage 3

Opik

see what your agent is doing.
find out if it's any good.

we capture every prompt

What we trace: Step 1

PROMPT	"Check Ohio for Praxis updates"
TOOL CALL	
RETRIEVAL	
TOKENS	
LATENCY	

and which skill it picked

Step 2: tool calls

PROMPT	"Check Ohio for Praxis updates"
TOOL CALL	skill: check-compliance-updates(state=Ohio)
RETRIEVAL	
TOKENS	
LATENCY	

what data came back

Step 3: retrieval

PROMPT	"Check Ohio for Praxis updates"
TOOL CALL	skill: check-compliance-updates(state=Ohio)
RETRIEVAL	fetched 3 URLs, hashed, diffed against snapshots
TOKENS	
LATENCY	

what it cost you

Step 4: token cost

PROMPT	"Check Ohio for Praxis updates"
TOOL CALL	skill: check-compliance-updates(state=Ohio)
RETRIEVAL	fetched 3 URLs, hashed, diffed against snapshots
TOKENS	1,247 in · 312 out · $0.0089
LATENCY	

and how long it all took

Step 5: latency. now you have a trace.

PROMPT	"Check Ohio for Praxis updates"
TOOL CALL	skill: check-compliance-updates(state=Ohio)
RETRIEVAL	fetched 3 URLs, hashed, diffed against snapshots
TOKENS	1,247 in · 312 out · $0.0089
LATENCY	2.4s end-to-end

It's a plugin.

zero code changes. drop it in, restart the gateway, done.

```
$ openclaw plugins install @opik/opik-openclaw

# .env
OPIK_API_KEY=sk-...
OPIK_WORKSPACE=your-comet-username

$ docker compose restart openclaw-gateway
```

now every Telegram message your bot answers shows up as a trace in Opik.

and when a metric tanks, here's where to look

Triage table

Answer Relevance ↓	tighten the system prompt; check skill-description routing
Hallucination ↑	agent is filling gaps — add 'I don't have that' to the prompt
Context Precision ↓	retrieval is messy — tune filters or reranking

one more thing

"Autonomous" doesn't mean "unaccountable."

- agents run in Docker — bounded fs, bounded network
- secrets in .env, never in source
- the agent fetches and hashes — it never writes to monitored systems
- every turn is captured in Opik
- evals run on a schedule — drift gets caught
- skills are explicit Markdown files; no skill = no capability

okay you have the keys

Now go build it.

hash a page. diff a snapshot. trust the trace.
the agent watches the world for you.

REPO

github.com/saskinosie/odsc-2026-...

OPENCLAW

github.com/openclaw/openclaw

OPIK

comet.com/opik

Lesson 3 Observability

comet

ODSC AI EAST 2026

ODSC AI

ODSC AI

ODSC AI

AI X

LEADERSHIP SUMMIT

ODSC AI

EAST 2026

AI X LEADERSHIP SUMMIT

ODSC AI EAST 2026